

Annexe : comprendre Held–Karp à 100 %

Programmation dynamique et principe de Bellman, appliqués au TSP

Document autonome — aucun prérequis. On part de zéro, on démontre, on déroule des exemples complets, on s'entraîne sur des exercices corrigés.

Brahim AMSEGT — candidat n°13530 — CPGE MP — session 2026

Ce document n'a qu'un but : que tu puisses *expliquer Held–Karp au tableau* et répondre à n'importe quelle question dessus. On construit chaque idée à partir de rien (récursivité, mémorisation, ensembles, codage binaire, principe de Bellman) *avant* d'arriver à Held–Karp. Les encadrés bleus sont les **démonstrations** à savoir refaire ; les encadrés rouges, les **questions de jury**.

Table des matières

1	Le problème, en deux phrases	2
2	Brique 1 — la récursivité et le gâchis des recalculs	2
2.1	Un exemple sans prérequis : Fibonacci	2
2.2	La solution : mémoriser (retenir ce qu'on a déjà calculé)	3
3	Brique 2 — sous-ensembles et codage binaire	3
3.1	Ensembles et sous-ensembles	3
3.2	Représenter un sous-ensemble par un entier : le « bitmask »	3
4	Brique 3 — le principe d'optimalité de Bellman	4
5	Held–Karp : Bellman appliqué au TSP	4
5.1	L'état : que mémorise-t-on ?	4
5.2	La récurrence, <i>dérivée</i> du principe de Bellman	5
5.3	Dans quel ordre calculer ? Par taille de S croissante	5
6	Exemple pratique entièrement déroulé ($n = 4$)	6
7	Démonstrations rassemblées	7
7.1	Correction	7
7.2	Complexité en temps : $O(n^2 2^n)$	7
7.3	Complexité en mémoire : $O(n 2^n)$, et le mur à $n \approx 20$	7
7.4	Comparaison avec la force brute	7
8	Le code Python complet et commenté	7
9	Pour situer Held–Karp : d'autres applications de Bellman	8
10	Exercices corrigés	8
11	Antisèche pour l'oral	9
12	Glossaire	9

1 Le problème, en deux phrases

On a n points à visiter (les points de collecte), un point de départ fixé (le dépôt, numéroté 0), et un coût $w(i, j)$ pour aller de i à j . On cherche l'ordre de visite qui minimise le coût total d'un **cycle** passant une fois par chaque point et revenant au dépôt : c'est le *problème du voyageur de commerce* (TSP).

Intuition

Il y a $(n - 1)!/2$ cycles possibles. Dès $n = 15$, c'est plus de 43 milliards : impossible de tous les essayer. Held-Karp ne les essaie *pas* tous ; il calcule l'optimum en réutilisant intelligemment des calculs intermédiaires. Pour comprendre comment, il faut d'abord comprendre une idée beaucoup plus simple, sur un exemple sans aucun rapport avec les graphes : la suite de Fibonacci.

2 Brique 1 — la récursivité et le gâchis des recalculs

2.1 Un exemple sans prérequis : Fibonacci

La suite de Fibonacci est définie par $F(0) = 0$, $F(1) = 1$, et $F(n) = F(n - 1) + F(n - 2)$. La traduction directe en programme *récuratif* (une fonction qui s'appelle elle-même) est :

```
1 def fib(n):
2     if n < 2:
3         return n
4     return fib(n - 1) + fib(n - 2) # la fonction s'appelle elle-meme
```

Ça marche, mais c'est désastreux. Regarde l'**arbre des appels** pour $F(5)$ (figure 1) : pour calculer $F(5)$ il faut $F(4)$ et $F(3)$; pour $F(4)$ il faut $F(3)$ et $F(2)$... et le même $F(3)$ est recalculé *plusieurs fois*, $F(2)$ encore plus souvent.

Calcul NAÏF de $F(5)$: l'arbre d'appels recalcule sans cesse les mêmes valeurs

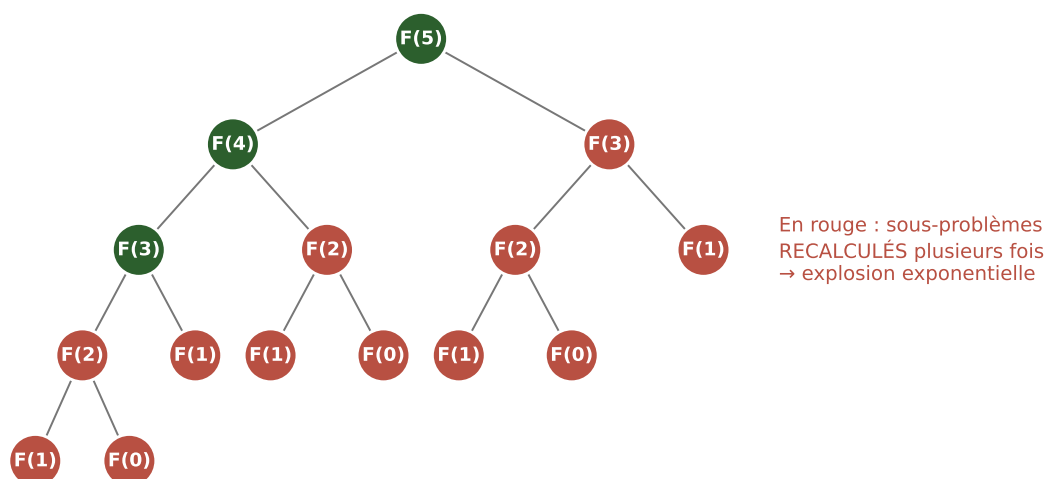


Figure 1: Calcul naïf de $F(5)$. Les sous-problèmes en rouge sont recalculés plusieurs fois. Le nombre total d'appels croît *exponentiellement* (environ $1,6^n$).

2.2 La solution : mémoriser (retenir ce qu'on a déjà calculé)

L'idée tient en une phrase : **calculer chaque sous-problème une seule fois, puis ranger le résultat dans un tableau pour le réutiliser**. C'est la *mémorisation*.

```
1 def fib(n, memo={}):
2     if n < 2:
3         return n
4     if n in memo: # déjà calculé ? on le relit, on ne recalcule pas
5         return memo[n]
6     memo[n] = fib(n - 1, memo) + fib(n - 2, memo)
7     return memo[n]
```

À retenir

Naïf : nombre d'appels $\sim 1,6^n$ (exponentiel). Mémorisé : chaque $F(k)$ pour $k = 2, \dots, n$ calculé *une fois*, soit $\Theta(n)$ opérations. On est passé d'exponentiel à linéaire *sans changer le résultat*, juste en arrêtant de recalculer. **C'est exactement ce que fait Held–Karp pour le TSP.**

Cette technique — décomposer en sous-problèmes qui se chevauchent, et mémoriser — s'appelle la **programmation dynamique**. Il nous manque deux briques pour l'appliquer au TSP : savoir manipuler des *sous-ensembles*, et le *principe de Bellman*.

3 Brique 2 — sous-ensembles et codage binaire

3.1 Ensembles et sous-ensembles

Un **ensemble** est une collection d'éléments, par exemple $V = \{0, 1, 2, 3\}$. Un **sous-ensemble** est une partie de cet ensemble : $\{1, 3\}$, $\{2\}$, ou même l'ensemble vide $\emptyset$. Un ensemble à m éléments possède exactement 2^m sous-ensembles (chaque élément est, indépendamment, *dedans* ou *dehors* : $2 \times 2 \times \dots = 2^m$).

3.2 Représenter un sous-ensemble par un entier : le « bitmask »

Intuition

Un ordinateur manipule mal des accolades. Astuce : on code un sous-ensemble par un *nombre binaire*, où le k -ième chiffre (bit) vaut 1 si l'élément k est dans l'ensemble, 0 sinon.

Exemple avec $V = \{0, 1, 2, 3\}$ (les bits se lisent de droite à gauche, position 0 à droite) :

$$\{1, 3\} \longrightarrow \underbrace{1}_3 \underbrace{0}_2 \underbrace{1}_1 \underbrace{0}_0 = 1010_2 = 10.$$

Les trois opérations dont Held–Karp a besoin (et leur écriture Python) :

Opération	Maths	Python
le singleton $\{k\}$	—	<code>1 << k</code>
tester « j est-il dans S ? »	$j \in S$	<code>S & (1 << j)</code>
retirer j de S	$S \setminus \{j\}$	<code>S & ~(1 << j)</code>

(« décale les bits, `&` est le « et » bit à bit, `~` inverse les bits.) Tu n'as pas besoin d'en savoir plus : retiens que *le sous-ensemble S est juste un entier*.

4 Brique 3 — le principe d’optimalité de Bellman

C’est le théorème qui *légitime* toute la programmation dynamique. Richard Bellman l’énonce en 1957.

Définition — principe d’optimalité (Bellman, 1957)

Dans un trajet optimal (de coût minimal), **tout sous-trajet est lui-même optimal** entre ses propres extrémités.

Intuition

Si le plus court chemin de Meknès à Tanger passe par Rabat, alors la portion Meknès→Rabat de ce chemin est forcément le plus court chemin de Meknès à Rabat. Sinon, on pourrait la remplacer par un trajet plus court et raccourcir le tout — contradiction.

Démonstration — le principe d’optimalité

Soit P un trajet optimal de a à b , et soit c un point intermédiaire de P . Notons P_1 la portion $a \rightarrow c$ et P_2 la portion $c \rightarrow b$, de sorte que $\text{coût}(P) = \text{coût}(P_1) + \text{coût}(P_2)$.

Supposons par l’absurde que P_1 ne soit *pas* optimal de a à c : il existerait un trajet P'_1 de a à c avec $\text{coût}(P'_1) < \text{coût}(P_1)$. En recollant P'_1 puis P_2 , on obtiendrait un trajet $a \rightarrow b$ de coût $\text{coût}(P'_1) + \text{coût}(P_2) < \text{coût}(P_1) + \text{coût}(P_2) = \text{coût}(P)$, donc strictement meilleur que P . Or P est optimal : contradiction. Donc P_1 est optimal. ■

À retenir

Cet argument s’appelle le « couper-coller » (*cut and paste*). C’est lui qui justifie qu’on peut construire une solution optimale en **recollant des sous-solutions optimales** — exactement ce que fait la récurrence de Held–Karp.

5 Held–Karp : Bellman appliqué au TSP

5.1 L’état : que mémorise-t-on ?

Le départ 0 est fixé une fois pour toutes ; on ne le « suit » donc pas. On s’intéresse aux sous-ensembles S de $\{1, 2, \dots, n-1\}$.

Définition — état de Held–Karp

Pour $S \subseteq \{1, \dots, n-1\}$ et $j \in S$,

$C(S, j)$ = coût minimal d’un trajet partant de 0, visitant exactement $\{0\} \cup S$, et finissant en j .

Intuition

Au lieu de demander « quel est le meilleur *tour complet* » (trop de réponses d’un coup), on demande une foule de petites questions : « quel est le meilleur chemin qui visite *ce groupe précis* de points et s’arrête *ici* ? ». On répond d’abord pour les petits groupes, puis on construit les grands à partir des petits.

5.2 La récurrence, *dérivée* du principe de Bellman

Démonstration — la récurrence de Held–Karp

Considérons un trajet optimal réalisant $C(S, j)$: il part de 0, visite $\{0\} \cup S$, finit en j . Juste *avant* j , il se trouvait en un certain sommet $i \in S, i \neq j$. La portion allant de 0 jusqu'à i visite alors exactement $\{0\} \cup (S \setminus \{j\})$ et finit en i . Par le principe d'optimalité, cette portion est elle-même optimale : son coût vaut donc $C(S \setminus \{j\}, i)$. Le coût total est alors $C(S \setminus \{j\}, i) + w(i, j)$. Comme on ignore quel est le bon prédécesseur i , on les essaie tous et on garde le meilleur :

$$C(S, j) = \min_{i \in S \setminus \{j\}} \left[C(S \setminus \{j\}, i) + w(i, j) \right]$$

Cas de base ($S = \{j\}$, trajet direct $0 \rightarrow j$) : $C(\{j\}, j) = w(0, j)$.

Fermeture du cycle (on revient au dépôt depuis le dernier sommet) :

$$\text{coût optimal du tour} = \min_{j \in \{1, \dots, n-1\}} \left[C(\{1, \dots, n-1\}, j) + w(j, 0) \right]. \quad \blacksquare$$

5.3 Dans quel ordre calculer ? Par taille de S croissante

$C(S, j)$ dépend de valeurs $C(S \setminus \{j\}, \cdot)$ dont l'ensemble a *un* élément de moins. Il suffit donc de remplir les coûts **par taille de S croissante** : d'abord les singletons (cas de base), puis les paires, puis les triplets, etc. (figure 2).

Held-Karp remplit les coûts $C(S, j)$ par taille de S CROISSANTE
(chaque sous-ensemble se déduit de ceux du niveau inférieur)

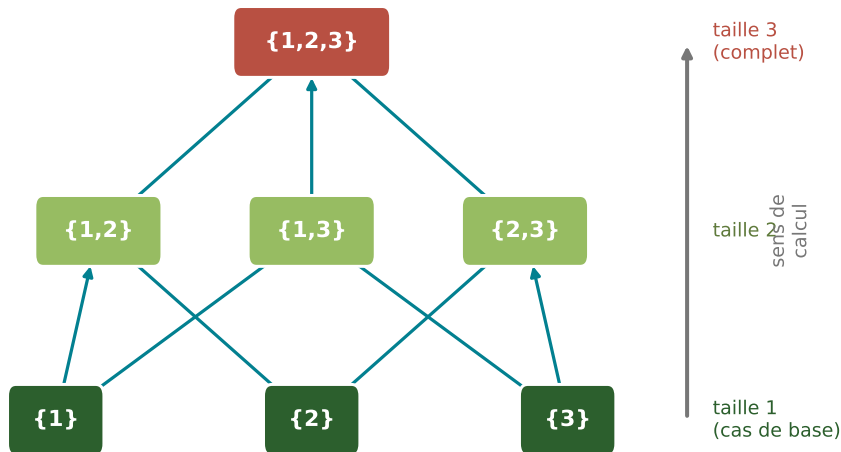


Figure 2: L'ordre de calcul : chaque sous-ensemble se déduit de ceux du niveau immédiatement inférieur. On monte le treillis, niveau par niveau.

Held-Karp : chaque coût $C(S, j)$ est calculé UNE fois
puis réutilisé (mémoïsation = principe de la prog. dynamique)

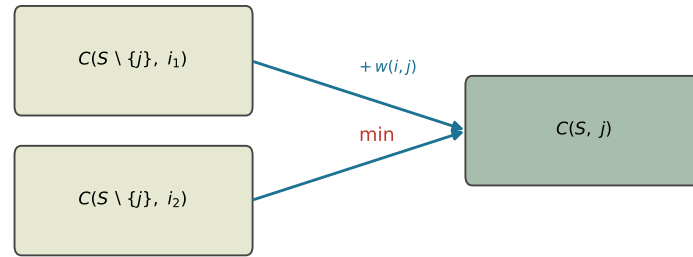


Figure 3: Chaque coût $C(S, j)$ est obtenu en prenant le *minimum*, sur tous les prédécesseurs possibles i , de $C(S \setminus \{j\}, i) + w(i, j)$. Calculé une fois, puis mémorisé.

6 Exemple pratique entièrement déroulé ($n = 4$)

Prenons quatre sommets $0, 1, 2, 3$ et la matrice des coûts (symétrique) :

	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

Étape 1 — cas de base ($C(\{j\}, j) = w(0, j)$) :

$$C(\{1\}, 1) = 10, \quad C(\{2\}, 2) = 15, \quad C(\{3\}, 3) = 20.$$

Étape 2 — sous-ensembles de taille 2 (un seul prédécesseur possible chacun) :

$$\begin{aligned} C(\{1, 2\}, 2) &= C(\{1\}, 1) + w(1, 2) = 10 + 35 = 45, & C(\{1, 2\}, 1) &= 15 + 35 = 50, \\ C(\{1, 3\}, 3) &= C(\{1\}, 1) + w(1, 3) = 10 + 25 = 35, & C(\{1, 3\}, 1) &= 20 + 25 = 45, \\ C(\{2, 3\}, 3) &= C(\{2\}, 2) + w(2, 3) = 15 + 30 = 45, & C(\{2, 3\}, 2) &= 20 + 30 = 50. \end{aligned}$$

Étape 3 — ensemble complet $\{1, 2, 3\}$ (deux prédécesseurs possibles, on prend le min) :

$$\begin{aligned} C(\{1, 2, 3\}, 1) &= \min [C(\{2, 3\}, 2) + w(2, 1), C(\{2, 3\}, 3) + w(3, 1)] = \min[50 + 35, 45 + 25] = \mathbf{70}, \\ C(\{1, 2, 3\}, 2) &= \min [C(\{1, 3\}, 1) + w(1, 2), C(\{1, 3\}, 3) + w(3, 2)] = \min[45 + 35, 35 + 30] = \mathbf{65}, \\ C(\{1, 2, 3\}, 3) &= \min [C(\{1, 2\}, 1) + w(1, 3), C(\{1, 2\}, 2) + w(2, 3)] = \min[50 + 25, 45 + 30] = \mathbf{75}. \end{aligned}$$

Étape 4 — fermeture du cycle :

$$\min [70 + w(1, 0), 65 + w(2, 0), 75 + w(3, 0)] = \min[70 + 10, 65 + 15, 75 + 20] = \boxed{80}.$$

À retenir

Reconstruction du tour (on remonte les choix mémorisés). Le minimum 80 est atteint en finissant par 1 (depuis $C(\{1, 2, 3\}, 1) = 70$, obtenu « via 3 »), lui-même atteint depuis $C(\{2, 3\}, 3) = 45$ (« via 2 »), lui-même depuis $C(\{2\}, 2) = 15$. En remontant :

$$0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0, \quad \text{coût } 15 + 30 + 25 + 10 = 80.$$

Vérification : l'énumération des $3! = 6$ tours donne aussi 80. Held-Karp a trouvé l'optimum exact en ne calculant que 12 valeurs $C(S, j)$ au lieu d'énumérer les tours entiers.

7 Démonstrations rassemblées

7.1 Correction

La récurrence est correcte par le principe de Bellman (démonstration §5.2) : tout préfixe d'un trajet optimal est optimal, donc $C(S, j)$ est bien obtenu en recollant un sous-trajet optimal $C(S \setminus \{j\}, i)$ avec l'arête finale $w(i, j)$, minimisé sur i . Par récurrence sur la taille de S , toutes les valeurs sont exactes, donc la fermeture donne l'optimum global.

7.2 Complexité en temps : $O(n^2 2^n)$

Démonstration — le coût en temps

Nombre d'états. Un état est un couple (S, j) avec $j \in S \subseteq \{1, \dots, n-1\}$. En sommant $|S|$ sur tous les sous-ensembles de $\{1, \dots, n-1\}$ (qui a $n-1$ éléments), on obtient $(n-1) 2^{n-2}$ états — de l'ordre de $n 2^n$.

Travail par état. Calculer $C(S, j)$ demande un minimum sur au plus $n-2$ prédécesseurs, soit $O(n)$.

Total. $O(n 2^n) \times O(n) = O(n^2 2^n)$. ■

7.3 Complexité en mémoire : $O(n 2^n)$, et le mur à $n \approx 20$

Il faut stocker tous les états : $O(n 2^n)$ cases. C'est en général la *mémoire* qui sature en premier. Pour $n = 20$: $20 \cdot 2^{20} \approx 2 \times 10^7$ cases (gérable). Pour $n = 25$: $25 \cdot 2^{25} \approx 8 \times 10^8$ cases, soit plusieurs Go : *rédhibitoire*. D'où le rôle de Held–Karp comme *étalon* sur des instances ≤ 15 –20 sommets, et non comme solveur des 30–40 points réels.

7.4 Comparaison avec la force brute

La force brute énumère les $(n-1)!/2$ tours : $O(n!)$. Held–Karp est en $O(n^2 2^n)$. Pour $n = 20$: $19!/2 \approx 6 \times 10^{16}$ contre $20^2 \cdot 2^{20} \approx 4 \times 10^8$ — un gain de *huit ordres de grandeur*. Held–Karp reste exponentiel (le TSP est NP-difficile), mais « beaucoup moins » : 2^n croît bien plus lentement que $n!$.

8 Le code Python complet et commenté

```
1 def held_karp(D):
2     """Résolution EXACTE du TSP. Temps  $O(n^2 \cdot 2^n)$ , mémoire  $O(n \cdot 2^n)$ ."""
3     n = len(D)
4     C = {} #  $C(S, j) = (\text{cout}, \text{predecesseur})$ 
5     for j in range(1, n): # cas de base : trajets 0 -> j
6         C[(1 << j, j)] = (D[0][j], 0)
7     for size in range(2, n): # on remplit par taille de S croissante
8         for subset in subsets(range(1, n), size):
9             bits = sum(1 << k for k in subset) # le sous-ensemble S, en binaire
10            for j in subset:
11                prev = bits & ~(1 << j) # S prive de j
12                C[(bits, j)] = min( # equation de Bellman :
13                    (C[(prev, i)][0] + D[i][j], i)
14                    for i in subset if i != j) # min sur les predecesseurs i
15    full = (1 << n) - 2 # tous les sommets sauf 0
16    cost, parent = min((C[(full, j)][0] + D[j][0], j) for j in range(1, n))
17    # reconstruction du tour en remontant les predecesseurs memorises
18    tour, bits, j = [0], full, parent
19    for _ in range(n - 1):
```

```

20     tour.append(j)
21     bits, j = bits & ~(1 << j), C[(bits, j)][1]
22     return tour[::-1], cost

```

9 Pour situer Held–Karp : d’autres applications de Bellman

Held–Karp n’est pas un truc isolé : c’est *un cas* d’une méthode très générale. Toutes ces célèbres méthodes reposent sur la même structure (sous-problèmes optimaux recollés) :

Bellman–Ford (plus courts chemins).

Calcule les plus courts chemins depuis une source, *même avec des poids négatifs*, en $O(VE)$. La valeur d’un chemin se met à jour par $d(v) = \min_u [d(u) + w(u, v)]$ — l’équation de Bellman, encore elle.

Floyd–Warshall (tous les plus courts chemins).

$O(V^3)$. *Utile pour toi* : c’est typiquement ce qui produit une matrice de distances point-à-point dans un réseau.

Sac à dos 0/1.

Maximiser la valeur emportée sous contrainte de poids, en $O(nW)$, en décidant objet par objet « je le prends ou pas ».

Distance d’édition (Levenshtein).

Nombre minimal d’opérations pour transformer un mot en un autre, en $O(mn)$; au cœur des correcteurs orthographiques.

10 Exercices corrigés

Exercice 1. Combien y a-t-il d’états $C(S, j)$ à calculer pour $n = 5$? Et pour $n = 10$?

Exercice 2. Sur la matrice de la section 6, calcule $C(\{1, 2\}, 2)$ et $C(\{1, 3\}, 3)$ *sans* regarder le corrigé de la section, en partant des cas de base.

Exercice 3. Déroule entièrement Held–Karp sur la matrice $w = \begin{bmatrix} 0 & 4 & 8 & 9 \\ 4 & 0 & 6 & 7 \\ 8 & 6 & 0 & 5 \\ 9 & 7 & 5 & 0 \end{bmatrix}$ ($n = 4$). Donne le coût optimal *et* le tour.

Exercice 4. Prouve, par l’argument du « couper-coller », que tout sous-chemin d’un plus court chemin est lui-même un plus court chemin.

Exercice 5. On veut maintenant un *chemin* (et non un cycle) partant de 0 et finissant n’importe où, de coût minimal. Comment modifier la *seule* étape de fermeture de Held–Karp ? Applique ta réponse à la matrice de la section 6.

Corrigés

1. Le nombre d’états est $(n - 1) 2^{n-2}$. Pour $n = 5$: $4 \cdot 2^3 = \mathbf{32}$. Pour $n = 10$: $9 \cdot 2^8 = \mathbf{2304}$.
2. $C(\{1, 2\}, 2) = C(\{1\}, 1) + w(1, 2) = 10 + 35 = \mathbf{45}$; $C(\{1, 3\}, 3) = C(\{1\}, 1) + w(1, 3) = 10 + 25 = \mathbf{35}$.

3. Cas de base : $C(\{1\}, 1) = 4$, $C(\{2\}, 2) = 8$, $C(\{3\}, 3) = 9$. Après remplissage des tailles 2 et 3 puis fermeture, l'optimum est **24**, atteint par le tour $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ ($4 + 6 + 5 + 9 = 24$). (Vérifiable par les 6 tours.)
4. Soit P un plus court chemin de a à b passant par c , de portions P_1 ($a \rightarrow c$) et P_2 ($c \rightarrow b$). Si un chemin P'_1 de a à c était strictement plus court que P_1 , alors P'_1 suivi de P_2 serait un chemin $a \rightarrow b$ strictement plus court que P : absurde. Donc P_1 est un plus court chemin $a \rightarrow c$ (de même pour P_2). ■
5. On supprime simplement le terme de retour $w(j, 0)$: le coût du meilleur chemin est $\min_j C(\{1, \dots, n-1\}, j)$. Sur la matrice de la section 6 : $\min(70, 65, 75) = \mathbf{65}$ (chemin finissant en 2).

11 Antisèche pour l'oral

Question piège du jury

« **Pourquoi fixer le sommet de départ 0 ?** » Parce qu'un cycle peut commencer par n'importe lequel de ses sommets : fixer le départ évite de recompter n fois le même cycle et divise le travail. Le choix de 0 est arbitraire et n'influe pas sur l'optimum.

Question piège du jury

« **En quoi est-ce de la programmation dynamique ?** » Deux ingrédients réunis : (1) *sous-structure optimale* (principe de Bellman : un préfixe optimal d'un trajet optimal), (2) *sous-problèmes qui se chevauchent* (le même $C(S, j)$ sert à plein de tours). On les calcule une fois et on mémorise.

Question piège du jury

« **Held–Karp pour $n = 40$?** » Non : $\sim 4 \times 10^{13}$ cases mémoire. Réservé aux instances $\leq 15-20$, où il sert d'**étalon** pour mesurer l'écart des heuristiques (plus proche voisin, 2-opt) à l'optimum exact.

12 Glossaire

Récurtivité

fonction qui s'appelle elle-même sur des cas plus petits.

Mémoïsation

ranger les résultats déjà calculés pour ne jamais les recalculer.

Programmation dynamique

résoudre en combinant des sous-problèmes optimaux mémorisés.

Principe d'optimalité (Bellman)

tout sous-trajet d'un trajet optimal est optimal.

Sous-structure optimale

propriété d'un problème dont l'optimum se construit à partir d'optima de sous-problèmes.

Bitmask

entier dont les bits codent l'appartenance à un sous-ensemble.

État $C(S, j)$

coût min d'un trajet $0 \rightarrow \dots \rightarrow j$ visitant exactement $\{0\} \cup S$.

NP-difficile

classe de problèmes sans algorithme polynomial connu (TSP en fait partie).

Document de travail personnel — la date du principe d'optimalité (Bellman, 1957) et celle de l'algorithme (Held & Karp, 1962) sont à reconfirmer dans les sources avant la soutenance.